discussed. If you need many endpoint-specific operations, you might consider using multiple endpoint filters.

As you saw in figure 5.8, adding multiple filters to an endpoint builds up a pipeline. Like the middleware pipeline, the endpoint filter pipeline can execute code both before and after the rest of the pipeline executes. Similarly, the filter pipeline can short-circuit in the same way as the middleware pipeline by returning a result and not calling `next`.

NOTE You've already seen an example of a short circuit in the filter pipeline. In listing 5.9 we short-circuit the pipeline if the `id` is invalid by returning a Problem Details object instead of calling `next(context)`.

As with middleware, the order in which you add filters to the endpoint filter pipeline is important. The filters you add first are called first in the pipeline, and filters you add last are called last. On the return journey through the pipeline, after the endpoint handler is invoked, the filters are called in reverse order, as with the middleware pipeline. As an example, consider the following listing, which adds an extra filter to the endpoint shown in listing 5.9.

Listing 5.10 Adding multiple filters to the endpoint filter pipeline

```
WebApplicationBuilder builder = WebApplication.CreateBuilder(args);
WebApplication app = builder.Build();
var _fruit = new ConcurrentDictionary<string, Fruit>();

app.MapGet("/fruit/{id}", (string id) =>
    _fruit.TryGetValue(id, out var fruit)
        ? TypedResults.Ok(fruit)
        : Results.Problem(statusCode: 404))
    .AddEndpointFilter(ValidationHelper.ValidateId)                    ❶
    .AddEndpointFilter(async (context, next) =>                        ❷
    {
        app.Logger.LogInformation("Executing filter...");              ❸
        object? result = await next(context);                          ❹
```